Introduction to Deep Learning

Magda Gregorová

magda.gregorova@thws.de

March 2026

Licensed according to CC-BY-SA 4.0

Contents

I. Foundations 3

1. What is Deep Learning? 3
- 1.1. Demonstration . . . 3
- 1.2. What is Deep Learning? . . . 3
- 1.3. Brief History . . . 4
- 1.4. Why Now? . . . 6
  - 1.4.1. Data . . . 6
  - 1.4.2. Compute . . . 7
  - 1.4.3. Algorithms . . . 7
  - 1.4.4. Community . . . 8
- 1.5. What Can Deep Learning Do? . . . 9
  - 1.5.1. Language and Communication . . . 9
  - 1.5.2. Vision and Perception . . . 9
  - 1.5.3. Healthcare and Medicine . . . 9
  - 1.5.4. Generation . . . 9
  - 1.5.5. Scientific Discovery . . . 10
  - 1.5.6. Decision Making and Control . . . 10

2. Shallow Neural Networks 10
- 2.1. Supervised Learning . . . 10
- 2.2. Regression and the Linear Model . . . 11
- 2.3. Nonlinear Regression and Activation Functions . . . 12
- 2.4. Combining ReLUs: Piecewise Linear Functions . . . 13
- 2.5. 1-Hidden-Layer Neural Networks . . . 14
- 2.6. Universal Approximation . . . 15
- 2.7. Binary Classification . . . 16
- 2.8. Biological vs Artificial Neuron . . . 17

Acknowledgements 18

References **19**

Part I. Foundations

1. What is Deep Learning?

1.1. Demonstration

▹ Session 1, Slides 1-12

Before any theory, consider the following experiment. A large language model - specifically Claude - is prompted with a single sentence:

> *"Write me a PyTorch script that trains a neural network to classify CIFAR-10 images into 10 classes. Use a simple MLP architecture. Print the training loss each epoch and plot a few example predictions with their true and predicted labels at the end."*

The model produces a complete, runnable Python script. When executed, the script downloads the CIFAR-10 dataset - 60,000 colour images of 32×32 pixels across 10 classes - trains a neural network, and achieves reasonable classification accuracy. A second prompt requesting a CNN instead of an MLP produces a different script that achieves noticeably higher accuracy on the same task.

This demonstration works. It is genuinely easy. And therein lies the problem: *what just happened?*

The script contains weights, layers, a loss function, a training loop, gradient updates. None of these were explained. The goal of this course is to open that black box - not to prompt better, but to think deeper.

1.2. What is Deep Learning?

▹ Session 1: What is Deep Learning?

Artificial intelligence, machine learning, and deep learning are often used interchangeably in popular discourse, but they refer to distinct and nested concepts.

Artificial intelligence is the broadest term - it encompasses any technique that enables machines to exhibit behaviour we would consider intelligent, including rule-based expert systems, search algorithms, and learned models.

Machine learning is a subfield of AI in which systems learn from data rather than being programmed with explicit rules. Instead of writing down every decision, we provide examples and let the system discover the underlying patterns.

Deep learning is a subfield of machine learning that uses neural networks with many layers - so-called deep neural networks. The depth allows the network to learn increasingly abstract representations of the input data.

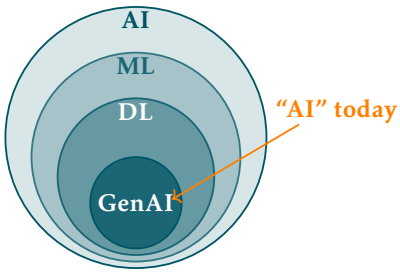

Figure 1: The relationship between AI, machine learning, deep learning, and generative AI.

Generative AI - encompassing large language models as well as image, video, and music generation systems - is a specific and currently prominent subset of deep learning. What the general public refers to as "AI" today is almost exclusively generative AI, which sits at the innermost level of this hierarchy.

Definition 1 (Deep Learning)**.** Deep learning is a class of machine learning methods that use multi-layered neural networks to learn hierarchical representations directly from raw data.

This definition will sharpen as the course progresses. For now it is sufficient to hold it loosely - by the end of the course every word in it will have a concrete meaning.

1.3. Brief History

▹ Session 1: A Brief History of Deep Learning

Figure 2 shows the key milestones in the development of deep learning alongside the periods of reduced funding and interest known as AI winters. For a comprehensive overview of the field's history see LeCun, Bengio, and G. Hinton 2015 and Goodfellow, Bengio, and Courville 2016.

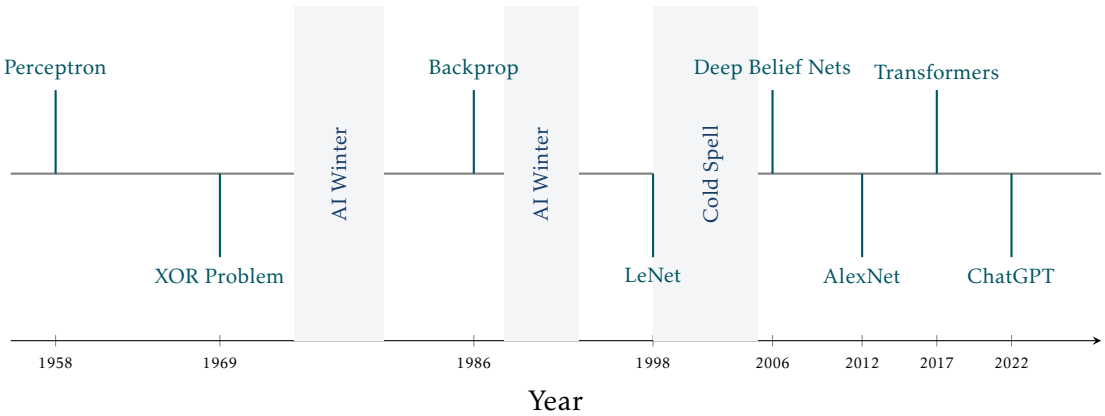

Figure 2: Key milestones in the history of deep learning. Shaded regions indicate AI winters and the cold spell of the early 2000s.

1958 - Perceptron. Frank Rosenblatt 1958 introduced the perceptron - the first trainable neural network. It could learn to classify linearly separable patterns by adjusting weighted connections between inputs and a single output unit. The result generated enormous excitement and equally enormous overpromising about the future of machine intelligence. Rosenblatt himself claimed the perceptron would eventually be able to walk, talk, see, write, reproduce itself, and be conscious of its existence - a level of optimism that would later contribute to the severity of the backlash.

The years following the perceptron saw genuine progress. The ADALINE model Widrow and Hoff 1960 introduced the least mean squares learning rule, which remains in use today. The concept of multilayer networks was understood theoretically, and researchers were actively exploring how to train them. There was a widespread belief that human-level artificial intelligence was only a decade away - a belief shared by many prominent scientists and echoed in generous research funding from DARPA and other agencies.

However, the limitations of single-layer networks were becoming apparent in practice. Many of the problems researchers wanted to solve - pattern recognition, language understanding, game playing - turned out to be far harder than anticipated. The gap between the promise of the perceptron and the reality of what it could achieve was widening.

1969 - XOR Problem. Minsky and Papert 1969 published *Perceptrons*, demonstrating formally that single-layer networks could not solve problems as simple as the XOR

function. The 1970s were a difficult period for neural network research. The Lighthill Report Lighthill 1973 delivered a damning assessment of AI research in 1973, concluding that the field had failed to deliver on its promises. DARPA cut funding significantly and many researchers moved away from neural networks entirely. This was the first AI winter.

Yet work continued quietly. Paul Werbos derived the backpropagation algorithm in his 1974 PhD thesis Werbos 1974 - years before it became widely known. Fukushima 1980 introduced the Neocognitron, a hierarchical neural network inspired by the visual cortex that anticipated many ideas later formalised in convolutional networks. Hopfield 1982 networks demonstrated that recurrent networks could function as associative memories. These contributions kept the field alive through the cold years, largely unnoticed by the mainstream AI community which had shifted its focus toward symbolic reasoning and expert systems.

Japan's role in this period deserves particular mention. Fukushima's Neocognitron emerged from NHK Science and Technology Research Laboratories in Tokyo. In 1982 the Japanese government launched the ambitious Fifth Generation Computer Project Ministry of International Trade and Industry 1982, investing heavily in AI hardware and parallel computing. The project caused significant alarm in the US and UK and triggered substantial counter-investment in AI research on both sides of the Atlantic. Its ultimate failure to deliver human-level reasoning contributed to the disillusionment of the second AI winter.

1986 - Backpropagation. Rumelhart, G. E. Hinton, and Williams 1986 popularised the backpropagation algorithm, providing a practical method for training multi-layer networks by propagating error gradients backwards through the layers. The revival of backpropagation generated enormous enthusiasm. Multilayer networks were successfully applied to speech recognition, image classification, and financial forecasting. LeCun applied backpropagation to convolutional networks at Bell Labs, demonstrating practical handwriting recognition systems used by the US postal service. Hochreiter and Schmidhuber introduced the Long Short-Term Memory (LSTM) network in 1997 Hochreiter and Schmidhuber 1997, solving the vanishing gradient problem for recurrent networks and laying the foundation for sequence modelling over the following two decades.

However, the second AI winter arrived in the late 1980s and early 1990s. Expert systems - rule-based AI programs that had attracted enormous commercial investment - collapsed as their maintenance costs proved prohibitive and their brittleness became apparent. The Lisp machine market crashed in 1987. DARPA again cut funding. Neural networks hit their own practical ceiling: training deep networks remained unstable, datasets were small, and support vector machines (SVMs) - introduced by Vapnik 1995 - offered better performance on many tasks with stronger theoretical guarantees. By the mid-1990s SVMs had largely displaced neural networks as the method of choice for classification problems.

1998 - LeNet. Yann LeCun, Bottou, et al. 1998 demonstrated convolutional neural networks for handwritten digit recognition, achieving state-of-the-art results on the MNIST dataset. Despite its success, neural networks remained out of favour through the early 2000s - a period sometimes called the cold spell - as support vector machines and other kernel methods dominated the field.

This cold spell was not a complete winter - small groups continued working on deep architectures. Bengio, Delalleau, and Le Roux 2004 explored the difficulty of training

deep networks, identifying the vanishing gradient problem as the central obstacle. Hinton's group at Toronto worked on unsupervised pretraining as a way to initialise deep networks before fine-tuning with backpropagation. The theoretical groundwork for the 2006 breakthrough was being laid quietly throughout this period.

2006 - Deep Belief Nets. G. E. Hinton, Osindero, and Teh 2006 showed that deep networks could be trained effectively using a layer-wise pretraining procedure . This reignited interest in deep architectures and marked the beginning of the modern deep learning era, even before large datasets and GPU training were fully established.

The key developments of this period were infrastructural rather than algorithmic. Raina, Madhavan, and Ng 2009 demonstrated that GPUs could accelerate neural network training by an order of magnitude, making experiments that previously took weeks feasible in hours. The ImageNet dataset was released by Deng et al. 2009, providing for the first time a benchmark large and challenging enough to reveal the true potential of deep architectures. Nair and G. E. Hinton 2010 introduced the ReLU activation function, which proved dramatically more effective than sigmoid and tanh activations for training deep networks. These three developments - GPU training, large-scale data, and better activations - set the stage for AlexNet's breakthrough in 2012.

2012 - AlexNet. Krizhevsky, Sutskever, and G. E. Hinton 2012 entered a deep convolutional network trained on GPUs into the ImageNet challenge and won by a margin that shocked the field - reducing the top-5 error rate from 26% to 15% in a single year. This moment is widely regarded as the turning point that established deep learning as the dominant paradigm in machine learning.

2017 - Transformers. Vaswani et al. 2017 introduced the transformer architecture in the paper *Attention Is All You Need*, replacing recurrent computation with self-attention mechanisms. The transformer became the foundation of virtually all modern large-scale models in both language and vision.

2022 - ChatGPT. OpenAI released ChatGPT, bringing large language models to a mass audience for the first time. Within two months it had reached 100 million users, making it the fastest-growing consumer application in history and triggering a wave of public and institutional interest in artificial intelligence that continues today.

The pattern visible in Figure 2 is worth noting: the history of deep learning is one of recurring cycles of excitement and disappointment, punctuated by genuine breakthroughs. The current era is distinguished not just by the scale of the results but by the breadth of their impact - for the first time, deep learning systems are changing how ordinary people work and communicate on a daily basis.

1.4. Why Now?

▹ Session 1: Why Now? - Four Driving Forces

Deep learning is not a new idea - neural networks have existed since the 1950s. What changed is the confluence of four factors that together made the approach viable at scale. None of these alone was sufficient; together they triggered a revolution.

1.4.1. Data

▹ Session 1: Why Now? - Four Driving Forces, Data - Scale Revolution

The internet created the largest dataset in history by accident. Every uploaded image, every written caption, every search query became potential training material. This was crystallised deliberately in benchmark datasets - ImageNet (2009) provided 1.2 million labelled images across 1,000 categories, and GLUE provided a standardised benchmark

for natural language understanding. These benchmarks did not just measure progress, they created it by giving the community a shared target to optimise against.

Crucially, the field also learned that labels are not always necessary. Self-supervised and weakly supervised learning techniques extract signal directly from unlabelled data, massively expanding what counts as usable training material. The web, in its entirety, became a dataset.

Figure 3 shows the growth in dataset scale over time. Two aspects deserve attention. First, the left axis is logarithmic - each gridline represents a tenfold increase, so the jump from ImageNet to LAION-5B represents roughly four thousand times more data. Second, the orange line shows that resolution grew alongside quantity - from 28×28 pixels for MNIST to 1024×1024 for LAION-5B, a 1,000-fold increase in pixels per image. The two dimensions of scale - how many images and how much information per image - grew together.

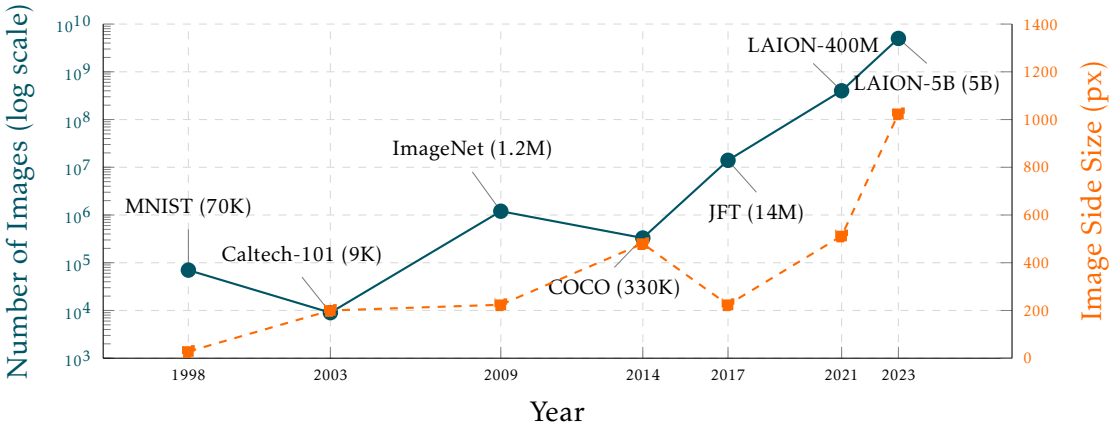

Figure 3: Image dataset sizes and resolutions over time. Blue line: number of images (log scale, left axis). Orange line: image side size in pixels (linear scale, right axis).

1.4.2. Compute

▹ Session 1: Compute & Progress

Graphics processing units, designed for the parallel computation of gaming graphics, turned out to be ideally suited for the matrix multiplications at the heart of neural network training. NVIDIA's CUDA programming model made GPUs accessible to researchers without hardware expertise. Custom chips - Google's TPUs, Apple Silicon, and a wave of AI-specific inference hardware - followed, with architectures designed around deep learning from the ground up.

Cloud platforms such as AWS, Microsoft Azure, and Google Cloud democratised access to large-scale compute - a researcher can now rent state-of-the-art hardware by the hour. Learning platforms such as Google Colab, Kaggle Notebooks, and Lightning.ai went further, providing free GPU access directly in the browser with zero setup. As Figure 4 shows, GPU performance has grown from single-digit TFLOPS in 2012 to nearly 2,000 TFLOPS in 2024.

1.4.3. Algorithms

▹ Session 1: Why Now? - Four Driving Forces, Compute & Progress

Several targeted innovations removed practical obstacles that had blocked progress for decades. At the architectural level, the multilayer perceptron (MLP), convolutional neu-

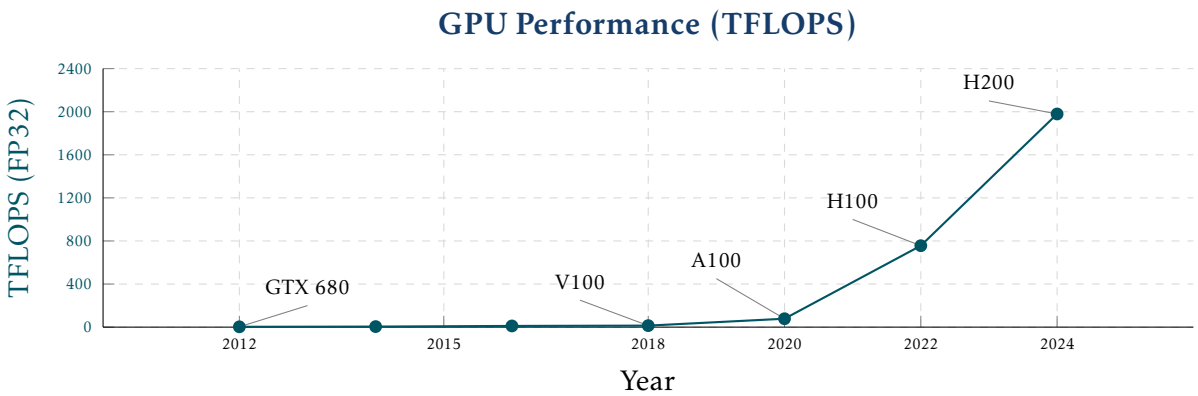

Figure 4: GPU performance in TFLOPS (FP32) from 2012 to 2024.

ral network (CNN), and recurrent neural network (RNN) established the foundational building blocks, all trained via the backpropagation algorithm. The ReLU activation function mitigated the vanishing gradient problem that had plagued deep networks. Batch normalisation stabilised training. Dropout provided effective regularisation.

At a higher level, residual connections (ResNets) made it possible to train networks of hundreds of layers. The attention mechanism and the transformer architecture, introduced in 2017, replaced recurrent computation entirely and became the foundation of modern language and vision models.

Figure 5 illustrates the impact of these algorithmic advances on the ImageNet benchmark. In 2011, the best classical methods using SVMs achieved around 74% top-5 accuracy. AlexNet's jump to 84.7% in 2012 marked the beginning of the deep learning era. Each subsequent architectural innovation - VGG, ResNet, EfficientNet, ViT - pushed accuracy further, surpassing human performance of approximately 95% around 2015 and reaching over 99% by 2022.

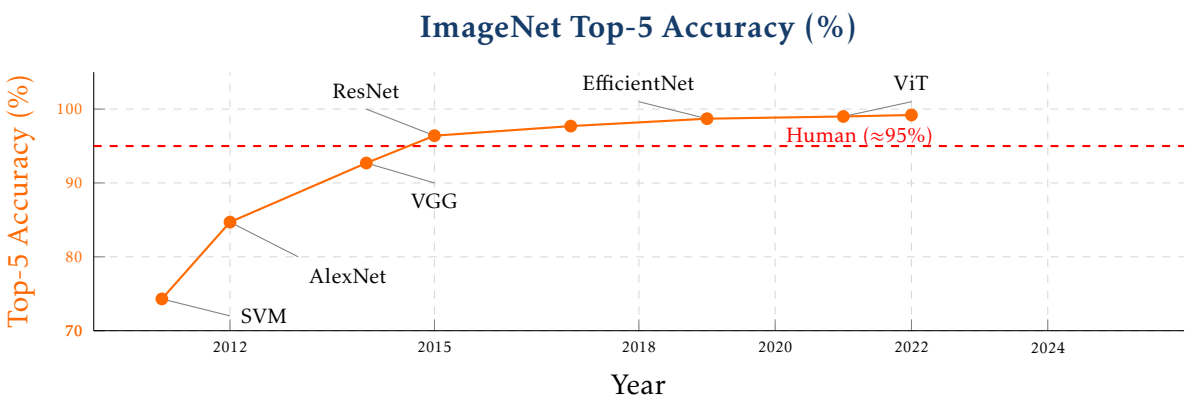

Figure 5: ImageNet top-5 accuracy from 2011 to 2022. Each labelled point corresponds to a key architectural innovation. The dashed red line indicates approximate human performance.

1.4.4. Community

Perhaps the most underappreciated factor is the open culture of the deep learning community. Frameworks such as PyTorch, TensorFlow, and JAX are open source,

actively maintained, and freely available. The norm of releasing code alongside papers - through platforms such as Papers with Code, Kaggle, and HuggingFace - means that results are reproducible and immediately buildable upon. Research is shared on arXiv before peer review, and major conferences such as NeurIPS, ICML, and ICLR publish proceedings openly.

This compounding effect - where each breakthrough immediately becomes everyone's foundation - has no parallel in most other scientific fields.

1.5. What Can Deep Learning Do?

▹ Session 1: What Can Deep Learning Do?

Deep learning today touches almost every domain of human activity. Rather than organising applications by the models used, it is more instructive to group them by the type of task they perform.

1.5.1. Language and Communication

Natural language was one of the earliest and remains one of the most impactful application areas. Machine translation systems such as DeepL and Google Translate now produce fluent output across dozens of language pairs, a task that resisted decades of rule-based approaches. Conversational agents - ChatGPT, Claude, Gemini - have brought language models to a mass audience, demonstrating capabilities in reasoning, summarisation, and question answering that were considered out of reach just a few years ago. Code generation tools such as GitHub Copilot and Cursor assist developers by completing, explaining, and generating code in real time.

1.5.2. Vision and Perception

Computer vision was the domain in which deep learning first demonstrated decisive superiority over classical methods, with AlexNet's 2012 ImageNet result marking the turning point. Today deep learning powers image classification, object detection, and semantic segmentation across a vast range of applications. Autonomous driving systems parse complex road scenes in real time, combining camera, lidar, and radar inputs to make driving decisions. Satellite and aerial imagery analysis enables large-scale monitoring of land use, deforestation, and infrastructure.

1.5.3. Healthcare and Medicine

Medical imaging has been transformed by deep learning. Systems trained on large collections of labelled scans can detect tumours, classify X-rays, and segment anatomical structures with accuracy approaching or exceeding that of specialist clinicians. Beyond imaging, deep learning is applied to genomics - identifying disease-associated variants in DNA sequences - and to clinical decision support systems that assist physicians with diagnosis and treatment planning.

1.5.4. Generation

Generative models have emerged as one of the most publicly visible applications of deep learning. Image generation systems such as Stable Diffusion and DALL-E produce photorealistic or stylised images from text descriptions. Video and music

generation - tools such as Suno and Udio - extend this capability to temporal media. The same underlying technology that enables generation also enables detection: deepfake detection systems attempt to identify synthetically generated media, an increasingly important capability as generation quality improves.

1.5.5. Scientific Discovery

Perhaps the most consequential application of deep learning to date is AlphaFold Jumper, Evans, Pritzel, et al. 2021, DeepMind's system for predicting protein three-dimensional structure from amino acid sequence. AlphaFold solved a fifty-year-old open problem in structural biology and earned Demis Hassabis and John Jumper the 2024 Nobel Prize in Chemistry - the first Nobel Prize awarded for work driven by deep learning. Deep learning is also applied to drug discovery - predicting molecular properties and identifying candidate compounds - and to climate and weather modelling, where neural networks are beginning to challenge traditional physics-based simulation approaches.

1.5.6. Decision Making and Control

Reinforcement learning - a branch of deep learning in which agents learn by interacting with an environment - has produced some of the field's most dramatic results. AlphaGo Silver, Huang, Maddison, et al. 2016 defeated the world champion Go player in 2016, a milestone considered a decade away by most experts at the time. Robotic systems learn manipulation and locomotion skills directly from experience, without hand-coded controllers. Recommendation systems - the algorithms that determine what appears in TikTok feeds, Netflix homepages, and Spotify playlists - are among the most economically impactful applications of deep learning, shaping the daily media consumption of billions of people.

2. Shallow Neural Networks

▹ Session 2

These notes accompany the second lecture of the course. The goal of this session is to build up, from first principles, the simplest possible neural network - a so-called **shallow** or **one-hidden-layer** neural network - and to understand precisely what it can represent. No optimisation is discussed here; that comes in later sessions. The question we ask is purely one of expressiveness: what class of functions can such a network compute?

2.1. Supervised Learning

▹ Session 2: Supervised Learning

We begin with the general framework that motivates everything that follows. In **supervised learning** we are given a dataset of input-output pairs

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$$

where each input $\mathbf{x}^{(i)} \in \mathcal{X}$ belongs to some input space $\mathcal{X}$ and each output $y^{(i)} \in \mathcal{Y}$ belongs to some output space $\mathcal{Y}$. We assume that there exists a true underlying relationship - a mapping $f^* : \mathcal{X} \to \mathcal{Y}$ such that $y = f^*(\mathbf{x})$, but we never have direct access to f^*. We only have access to the finitely many observations in the dataset $\mathcal{D}$.

The goal is to learn a parameterised function $f_\theta : \mathcal{X} \to \mathcal{Y}$ from $\mathcal{D}$ such that it approximates the true unkonwn mapping $f_\theta \approx f^*$. The notation $f_\theta(\mathbf{x}) = f(\mathbf{x};\theta) = \hat{y}$ will be used throughout the course: $\hat{y}$ denotes the model's prediction for input $\mathbf{x}$ given parameters θ. The goal is not merely to fit the training data, but to predict accurately on ***unseen*** inputs - this is the **generalisation** requirement, which we will revisit in depth in later sessions.

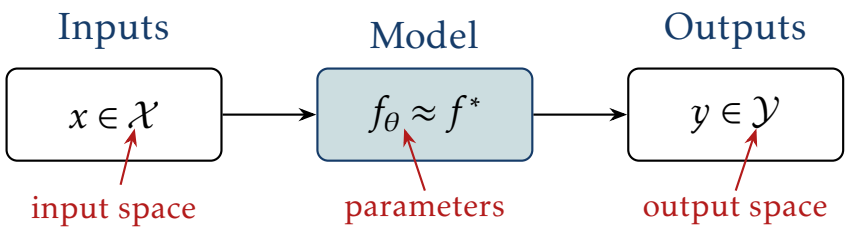

Figure 6: Supervised learning: given data pairs $(\mathbf{x}, y)$, learn a model f_θ that approximates the true relationship f^*.

Supervised learning encompasses two fundamental task types. In **regression**, the output space is continuous: $\mathcal{Y} = \mathbb{R}$ (or more generally $\mathbb{R}^k$), and the goal is to predict a numerical value. In **classification**, the output space is discrete, and the goal is to assign each input to one of two categories. These two tasks are illustrated in Figure 7 and Figure 8.

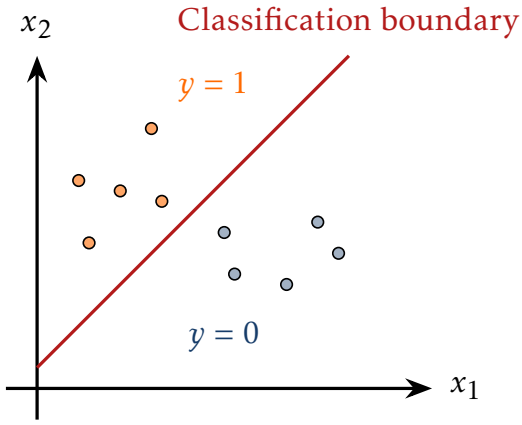

Figure 7: Binary classification: two classes separated by a decision boundary.

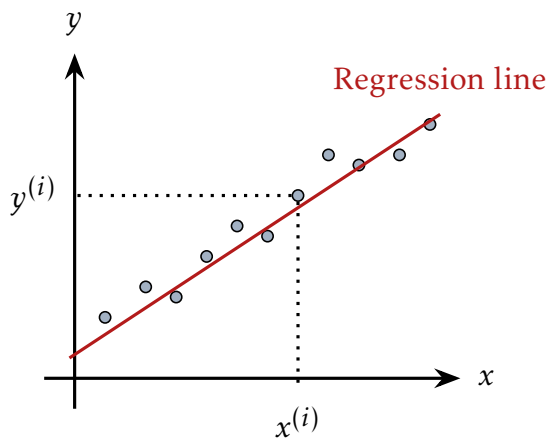

Figure 8: Regression: fitting a continuous function to data points.

2.2. Regression and the Linear Model

We focus first on the regression setting with a single input $x \in \mathbb{R}$ and a single output $y \in \mathbb{R}$.

▹ Session 2: Regression and Linear Model

The simplest parameterised function we will work with is an affine function - often called simply **linear model** in the machine learning literature:

$$\hat{y} = f_\theta(x) = \theta_1 x + \theta_0, \qquad \boldsymbol{\theta} = (\theta_0, \theta_1) \in \mathbb{R}^2.$$

This model has two parameters: a slope θ_1 and an intercept θ_0 and can be represented graphically as a single **neuron**, shown in Figure 9. The input x and the constant 1 (carrying the bias θ_0) feed into the neuron, which computes the weighted sum and produces the output $\hat{y}$. This graphical representation may seem unnecessary for such a simple model, but it will prove invaluable as we build more complex networks - the same diagram extends naturally to multiple inputs, multiple neurons, and multiple layers.

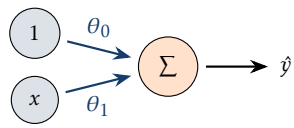

Figure 9: Graphical representation of the linear model as a single neuron.

The representational capacity of the linear model is severely limited - it can only fit data that lies approximately along a straight line. When the true relationship f^* is nonlinear, a linear model will fail systematically, as illustrated in Figure 11: it will underfit the data regardless of how the parameters are chosen. This is not a failure of optimisation; it is a failure of expressiveness. The model class is simply too small to contain the true function.

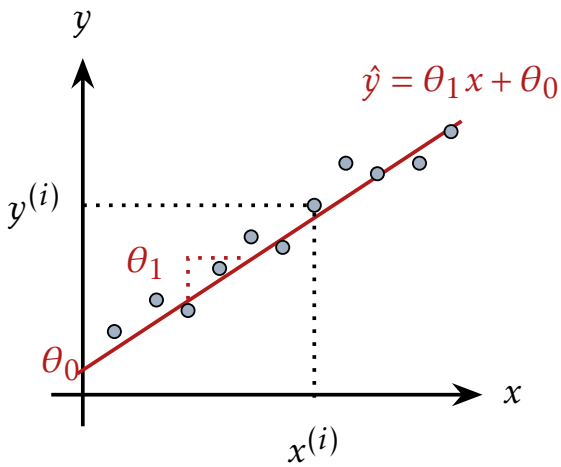

Figure 10: A linear model fits the data well when the true relationship is linear.

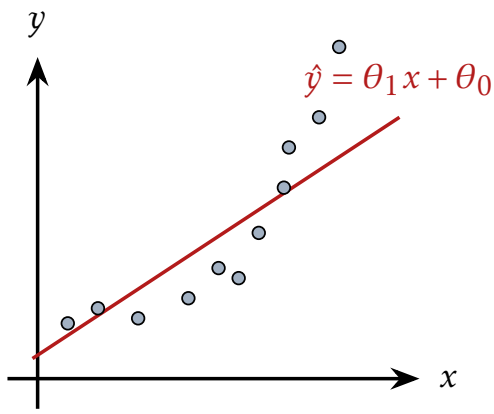

Figure 11: A linear model fails to capture nonlinear relationships in the data.

This observation motivates the central question of the lecture: how can we construct a richer class of functions f_θ that can capture nonlinear relationships?

2.3. Nonlinear Regression and Activation Functions

▸ Session 2: Nonlinear Regression

The linear model introduced in the previous section has a fundamental limitation: it can only represent affine relationships between inputs and outputs. To move beyond this, we introduce a **nonlinear activation function** $a : \mathbb{R} \to \mathbb{R}$ applied to the output of the linear computation. The neuron now computes:

$$\hat{y} = f_\theta(x) = a(\theta_1 x + \theta_0).$$

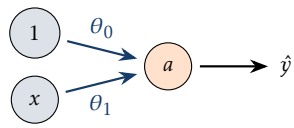

Figure 12: A nonlinear neuron applies activation function a to the linear combination of its inputs.

The activation function is what gives the neural network its expressive power -changing the mapping between inputs $\mathcal{X}$ and outputs $\mathcal{Y}$ from linear to nonlinear. The graphical representation of this nonlinear neuron is shown in

Figure 12: it is identical to the linear neuron, but the node now carries the symbol a to indicate that a nonlinearity is applied before the output is produced.

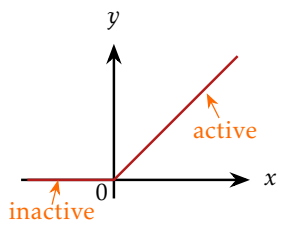

Figure 13: The ReLU activation function: inactive (zero) for $x \leq 0$, active (linear) for $x > 0$.

The choice of activation function matters. The most widely used activation function in modern deep learning is the **Rectified Linear Unit**, or **ReLU** (Nair and G. E. Hinton 2010; Glorot, Bordes, and Bengio 2011):

$$a(x) = \mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \quad \text{(active)} \\ 0 & x \leq 0 \quad \text{(inactive).} \end{cases}$$

The ReLU is remarkably simple: it passes positive values unchanged and clamps negative values to zero. A **ReLU neuron** is said to be **active** when its pre-activation value is positive, and **inactive** otherwise. Despite its simplicity - or perhaps because of it - the ReLU is extremely effective in practice and will be our default activation function throughout this course. We return to a broader comparison of activation functions in a later session.

Composing the ReLU with a linear function introduces exactly one breakpoint, at $x = -\theta_0/\theta_1$, where the output transitions from inactive (zero) to active (rising linearly). This single bend is still not expressive enough to capture complex nonlinear relationships. The key insight, developed in the next section, is that a *weighted sum* of such ReLU activated functions produces a piecewise linear function with one bend per ReLU - and piecewise linear functions can approximate any continuous function.

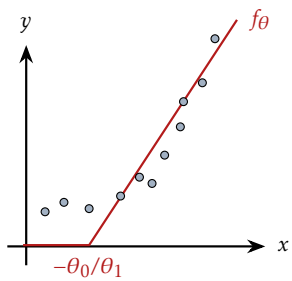

Figure 14: A single ReLU applied to a linear function produces one breakpoint at $x = -\theta_0/\theta_1$.

2.4. Combining ReLUs: Piecewise Linear Functions

▹ Session 2: Combining Multiple ReLUs

A single ReLU neuron introduces one breakpoint and is therefore only marginally more expressive than a linear model. The power comes from combining multiple such neurons. Consider a weighted sum of k ReLU-activated linear functions:

$$\hat{y} = f_\theta(x) = \theta_0 + \sum_{j=1}^{k} \theta_j \, a(\theta_{j1} x + \theta_{j0}).$$

Each term contributes one breakpoint at $x = -\theta_{j0}/\theta_{j1}$. The resulting function is **piecewise linear** - linear within each interval between breakpoints, with up to k bends in total. As k increases, the function can capture increasingly complex shapes, as illustrated in Figure 15.

The graphical representation of this computation for $k = 2$ ReLU activations is shown in Figure 16. Each **hidden neuron** j - so called because its activations are internal to the network and not directly observed as inputs or outputs - receives the input x and independently computes $h_j = a(\theta_{j1} x + \theta_{j0})$. Note that in the full representation (left), the input x and bias constant 1 are shown separately for each neuron — this makes

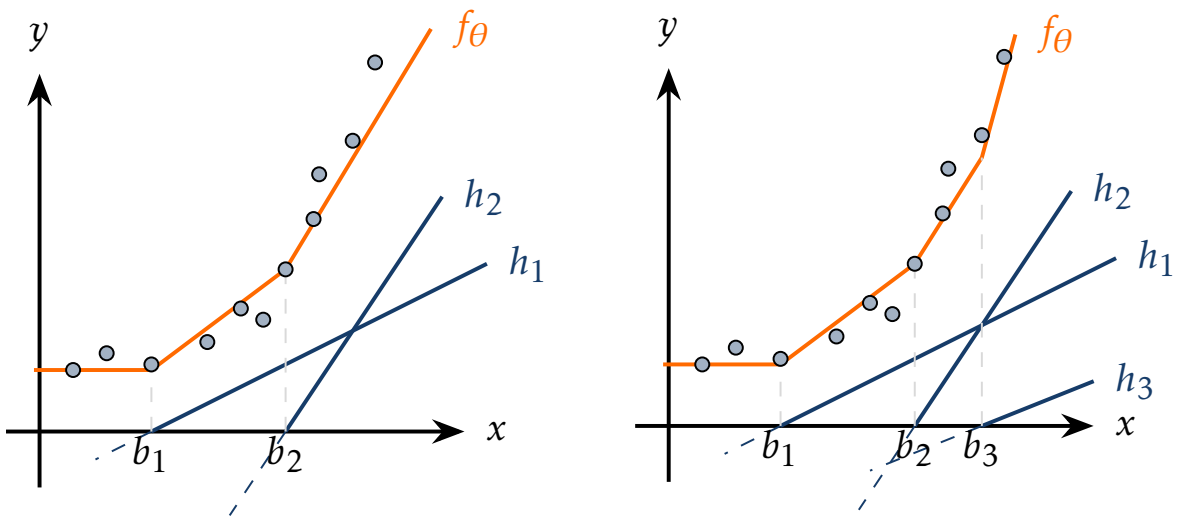

Figure 15: Left: combination of two ReLU functions producing two breakpoint. Right: combination of three ReLU functions producing three breakpoints. More neurons yield more expressive functions.

explicit that each neuron has its own independent set of parameters $(\theta_{j1}, \theta_{j0})$, even though they share the same input. The output neuron then forms the weighted sum $\hat{y} = \theta_0 + \theta_1 h_1 + \theta_2 h_2$. The centre diagram shows the same network with parameters omitted and the bias nodes drawn as dashed - a common convention indicating that bias inputs are always present but typically not drawn explicitly. The right diagram shows only the bare network topology. An important detail not visible in any of these diagrams: the output neuron o applies no activation function - it computes a plain weighted sum. Nonlinearity is introduced only in the hidden layer; the output is linear in the hidden activations.

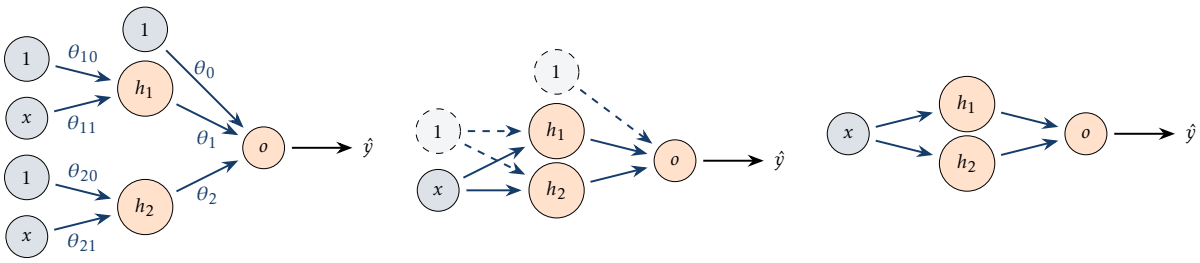

Figure 16: Three levels of abstraction for a two-neuron network. Left: full representation with all weights θ_{j1}, θ_{j0} and output weights labelled. Centre: simplified representation with bias nodes shown as dashed. Right: compact schematic showing topology only.

2.5. 1-Hidden-Layer Neural Networks

We now generalise from two hidden neurons to k hidden neurons, and from a single input to multiple inputs. The resulting architecture is a **one-hidden-layer neural network**, also called a **shallow neural network**.

▹ Session 2: One Layer Neural Network

With a single input $x \in \mathbb{R}$ and k hidden neurons, the network computes:

$$h_j = a(\theta_{j1}\, x + \theta_{j0}), \qquad j = 1, \ldots, k$$
$$\hat{y} = f_\theta(x) = \theta_0 + \sum_{j=1}^{k} \theta_j\, h_j.$$

Each hidden neuron j has its own parameters θ_{j1} (weight) and θ_{j0} (bias), and contributes one breakpoint to the piecewise linear output. The output neuron forms a weighted sum of all hidden activations with weights θ_j and a global bias θ_0. The graphical representation is shown in Figure 17.

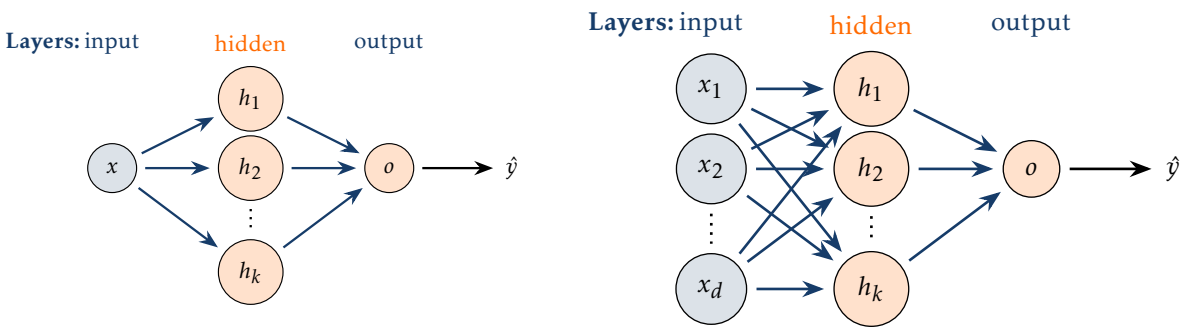

Figure 17: Left: one-hidden-layer network with a single input x and k hidden neurons. Right: extension to d inputs $\mathbf{x} = (x_1, \ldots, x_d)$. Each hidden neuron applies a ReLU activation; the output neuron computes a plain weighted sum.

Extending to d inputs $\mathbf{x} = (x_1, \ldots, x_d) \in \mathbb{R}^d$ is straightforward. Each hidden neuron now computes a linear combination of all inputs:

$$h_j = a\left(\sum_{i=1}^{d} \theta_{ji}\, x_i + \theta_{j0}\right), \qquad j = 1, \ldots, k$$
$$\hat{y} = f_\theta(\mathbf{x}) = \theta_0 + \sum_{j=1}^{k} \theta_j\, h_j.$$

The weight θ_{ji} connects input x_i to hidden neuron j, and θ_{j0} is the bias of neuron j. The output formula is unchanged — the hidden activations h_j are combined into a single prediction $\hat{y}$ regardless of the input dimensionality.

The collection of all parameters — the weights θ_{ji}, biases θ_{j0}, output weights θ_j, and output bias θ_0 — is denoted collectively as $\boldsymbol{\theta}$. Learning this network means finding the values of $\boldsymbol{\theta}$ that make $f_\theta(\mathbf{x}) \approx f^*(\mathbf{x})$ on the training data.

2.6. Universal Approximation

The piecewise linear intuition suggests that a shallow network with enough neurons can approximate any reasonable function. This is made precise by the **Universal Approximation Theorem**.

▹ Session 2: Universal Approximation

Theorem 1 (Universal Approximation (Cybenko 1989; Hornik 1991)). *For any continuous function $g : \mathcal{X} \to \mathbb{R}$ defined on a compact set $\mathcal{X} \subseteq \mathbb{R}^d$ and any $\varepsilon > 0$, there exists a one-hidden-layer neural network f_θ such that*

$$\sup_{\mathbf{x} \in \mathcal{X}} |f_\theta(\mathbf{x}) - g(\mathbf{x})| < \varepsilon.$$

This result was first proved by Cybenko 1989 for sigmoid activations and later extended by Hornik 1991 to a broad class of activation functions. We do not prove this result here as it relies on advanced math concepts from measure theory and the Stone-Weierstrass theorem. The piecewise linear intuition developed in the previous section - more neurons, more breakpoints, better approximation - provides the geometric essence of why the theorem holds.

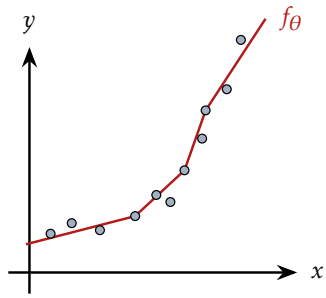

Figure 18: A piecewise linear function approximating a nonlinear curve.

The theorem is powerful: it says that the class of shallow neural networks is dense in the space of continuous functions. No matter what continuous function you want to approximate, a shallow network can get arbitrarily close. However, the theorem has important limitations that are easily overlooked. It guarantees existence - there *exists* a network that approximates g - but says nothing about how large that network must be, or how to find its parameters. In the worst case, exponentially many neurons may be required. The theorem also says nothing about generalisation: a network that perfectly approximates f^* on $\mathcal{X}$ may still fail on out-of-distribution inputs. Depth, which the theorem ignores, turns out to matter enormously in practice - deeper networks can represent the same functions far more efficiently than shallow ones, a point we return to later.

2.7. Binary Classification

We now turn to the classification setting. In **binary classification**, the output space is $\mathcal{Y} = \{0, 1\}$, and the goal is to assign each input $\mathbf{x} \in \mathcal{X}$ to one of two classes.

▹ Session 2: Binary Classification

The simplest approach is a **linear classifier**: predict class 1 if a linear function of the input exceeds zero, and class 0 otherwise:

$$\hat{y} = \mathbf{1}\left[\theta_0 + \sum_{i=1}^{d} \theta_i x_i \geq 0\right].$$

The set of points satisfying $\theta_0 + \sum_i \theta_i x_i = 0$ is a hyperplane in $\mathbb{R}^d$ - the **decision boundary**. The weight vector $(\theta_1, \ldots, \theta_d)$ is normal to this hyperplane, and θ_0 controls its offset from the origin. In two dimensions, the decision boundary is a line; in three dimensions, a plane; in general, a $(d-1)$-dimensional hyperplane.

A linear classifier can only represent **linearly separable** problems - those in which the two classes can be separated by a single hyperplane. Many real problems are not linearly separable. A simple example: consider two clusters of one class located at the top-left and bottom-right of the input space, with a cluster of the other class in the middle. No straight line can separate them.

The solution is exactly analogous to the regression case: replace the linear function with a neural network. The decision boundary $f_\theta(\mathbf{x}) = 0$ becomes a nonlinear surface, capable of carving out arbitrarily complex regions of the input space:

$$\hat{y} = \mathbf{1}[f_\theta(\mathbf{x}) \geq 0].$$

The same shallow network architecture used for regression - a linear combination of ReLU-activated linear functions - can serve as f_θ here. The universal approximation theorem applies equally to classification.

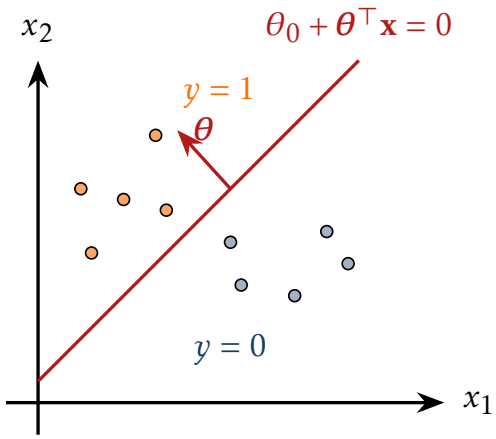

Figure 19: A linear classifier separates two classes with a hyperplane. The weight vector $\boldsymbol{\theta}$ is normal to the decision boundary.

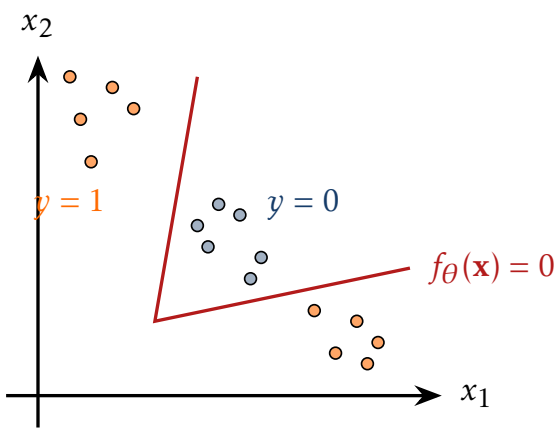

Figure 20: A nonlinearly separable dataset. A neural network with a nonlinear decision boundary $f_\theta(\mathbf{x}) = 0$ is required.

2.8. Biological vs Artificial Neuron

▹ Session 2: Biological vs Artificial Neuron

Having built up the neural network entirely from mathematical and computational considerations, it is worth pausing to examine the biological analogy that motivates the terminology.

A biological neuron illustrated in Figure 21 receives signals from other neurons via **dendrites**, integrates these signals in the **cell body** (soma), and fires an electrical pulse along its **axon** if the integrated signal exceeds a threshold. The artificial neuron in Figure 22 mimics this schematically: inputs $\mathbf{x}$ play the role of dendrites, the weighted sum $\sum_i \theta_i x_i + \theta_0$ plays the role of integration in the soma, and the activation function a plays the role of the firing threshold.

The analogy is useful as a narrative device but should not be taken literally. Real neurons are vastly more complex than the linear-threshold model: they operate in continuous time, communicate via spike trains rather than real-valued signals, exhibit rich temporal dynamics, and are embedded in three-dimensional physical structures that constrain their connectivity. The artificial neuron strips all of this away, retaining only the most abstract essence of the computation.

Indeed, the mathematical objects we have been working with - piecewise linear functions, weighted sums, nonlinear activations - were derived entirely from the regression and classification problems, without any reference to biology. The biological inspiration played a historical role in motivating the research programme, but the justification for using neural networks today is empirical and mathematical, not biological.

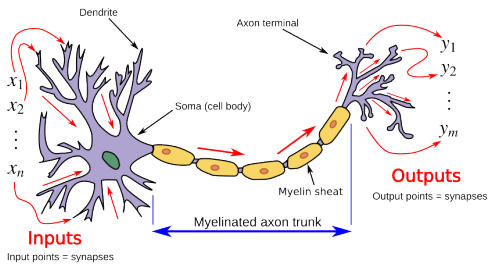

Image: Egm4313.s12 at English Wikipedia, CC BY-SA 3.0, via Wikimedia Commons

Figure 21: A biological neuron with dendrites, soma, and axon.

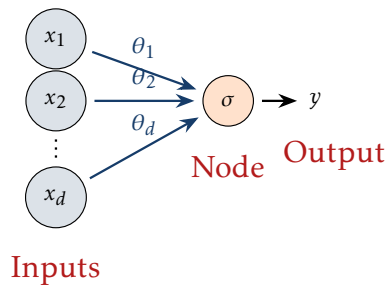

Figure 22: The artificial neuron: a weighted sum of inputs followed by a non-linear activation function.

Acknowledgements

These lecture notes were prepared with the assistance of Claude, a large language model developed by Anthropic. All content has been reviewed, edited, and approved by the author.

References

Bengio, Yoshua, Olivier Delalleau, and Nicolas Le Roux (2004). "The Curse of Highly Variable Functions for Local Kernel Machines". In: *Advances in Neural Information Processing Systems*.

Cybenko, George (1989). "Approximation by Superpositions of a Sigmoidal Function". In: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314. DOI: 10.1007/BF02551274.

Deng, Jia et al. (2009). "ImageNet: A Large-Scale Hierarchical Image Database". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

Fukushima, Kunihiko (1980). "Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position". In: *Biological Cybernetics* 36, pp. 193–202.

Glorot, Xavier, Antoine Bordes, and Yoshua Bengio (2011). "Deep Sparse Rectifier Neural Networks". In: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 315–323.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. Available at https://www.deeplearningbook.org. MIT Press.

Hinton, Geoffrey E., Simon Osindero, and Yee-Whye Teh (2006). "A Fast Learning Algorithm for Deep Belief Nets". In: *Neural Computation* 18.7, pp. 1527–1554.

Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780.

Hopfield, John J. (1982). "Neural Networks and Physical Systems with Emergent Collective Computational Abilities". In: *Proceedings of the National Academy of Sciences* 79.8, pp. 2554–2558.

Hornik, Kurt (1991). "Approximation Capabilities of Multilayer Feedforward Networks". In: *Neural Networks* 4.2, pp. 251–257. DOI: 10.1016/0893-6080(91)90009-T.

Jumper, John, Richard Evans, Alexander Pritzel, et al. (2021). "Highly Accurate Protein Structure Prediction with AlphaFold". In: *Nature* 596, pp. 583–589.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton (2012). "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton (2015). "Deep Learning". In: *Nature* 521, pp. 436–444.

LeCun, Yann, Léon Bottou, et al. (1998). "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.

Lighthill, James (1973). *Artificial Intelligence: A General Survey*. Tech. rep. Available at https://www.aiai.ed.ac.uk/events/lighthill1973/lighthill.pdf. Science Research Council.

Ministry of International Trade and Industry (1982). *Outline of Research and Development Plans for Fifth Generation Computer Systems*. Tech. rep. Institute for New Generation Computer Technology.

Minsky, Marvin and Seymour Papert (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

Nair, Vinod and Geoffrey E. Hinton (2010). "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the International Conference on Machine Learning*.

Raina, Rajat, Anand Madhavan, and Andrew Y. Ng (2009). "Large-Scale Deep Unsupervised Learning Using Graphics Processors". In: *Proceedings of the International Conference on Machine Learning*.

Rosenblatt, Frank (1958). "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6, pp. 386–408.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams (1986). "Learning Representations by Back-propagating Errors". In: *Nature* 323, pp. 533–536.

Silver, David, Aja Huang, Chris J. Maddison, et al. (2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529, pp. 484–489.

Vapnik, Vladimir (1995). *The Nature of Statistical Learning Theory*. Springer.

Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30.

Werbos, Paul (1974). "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences". PhD thesis. Harvard University.

Widrow, Bernard and Marcian E. Hoff (1960). "Adaptive Switching Circuits". In: *IRE WESCON Convention Record*. Vol. 4, pp. 96–104.